

Week 6. Problem set (solutions by Karim Khabibrakhmanov)

1. Apply COUNTING-SORT [CLRS, §8.2] to the following input array where each column corresponds to one item with its numeric key and single-character satellite data:

2	6	2	7	6	6	3	4	1	6	2	0	7
U	W	A	M	E	S	E	A	O	O	R	Y	E

You **must** demonstrate the final state of both auxiliary arrays used in the algorithm, as well as the final state of the output array.

Answer:

$Range = maxKey - minKey + 1 = 7 - 0 + 1 = 8$ — size of count table

The count table that stores the number of repetitions for each key

Keys	0	1	2	3	4	5	6	7
Amount of repetitions	1	1	3	1	1	0	4	2

The transformed count table, which stores the index for each occurrence from the array. Starting from the end of the array, we take the key, go to our transformed table, take out the index and insert it into a new already sorted array, while reducing the index by 1

Keys	0	1	2	3	4	5	6	7
Indexes before sorting	1	2	5	6	7	7	11	13
Indexes after sorting	0	1	2	5	6	7	7	11

The final sorted table:

Keys	0	1	2	2	2	3	4	6	6	6	6	7	7
Characters	Y	O	U	A	R	E	A	W	E	S	O	M	E

2. Explain how BUCKET-SORT [CLRS, §8.4] can be used to efficiently sort the points distributed uniformly¹ in a unit disk $\{(x, y) \mid x^2 + y^2 \leq 1\}$ by their distance² from the origin $(0, 0)$:

- (a) Explain the idea of splitting the points into buckets.

Answer:

To begin with, for each coordinate, we will calculate its distance from origin using the $\sqrt{x^2 + y^2}$. Since our points are in a circle, we can divide our circle into arcs, each with a range of $\frac{Radius = 1}{Amount\ of\ coordinates}$. In the process of writing coordinates to buckets for each distance, we will take its specific arc. This way we will have an array of buckets. Next, we will sort each bucket. At the end, we will write our sorted coordinates into the array.

- (b) Show how to assign each point to a bucket in $\Theta(1)$ time.

Answer:

- For each point, we calculate the distance using the formula $\sqrt{x^2 + y^2}$ - it takes $\Theta(1)$
- Next, we calculate the bucket index in which we put our coordinates - it also takes $\Theta(1)$
- And at the end we write our coordinates in the bucket - it is also $\Theta(1)$
- The result: $\Theta(1 + 1 + 1) = \Theta(1)$

¹Meaning that for each point the probability of being in any region R inside the disk is directly proportional to the area of that region: $P(R) = \frac{area(R)}{\pi}$.

²We consider the standard Euclidean distance $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

- (c) For each bucket, compute the probability of a random point to be assigned to it. Ensure that the sum of probabilities is 1.

Answer:

- Let's use the formula¹ $P(R) = \frac{\text{area}(R)}{\pi}$ given to us and calculate $\text{area}(R)$
- Each area has a range of $\frac{1}{\text{amountOfCoordinates} = n}$
- To calculate this area, we need to subtract the area of the circle along the outer arc from the area of the circle of the inner arc.
- Let k be the index of an arbitrary bucket, then the formula will look like:

$$\text{i. } \text{area}(R) = \pi \cdot r_{\text{out}}^2 - \pi \cdot r_{\text{in}}^2 = \pi \cdot \left(\frac{k^2}{n^2} - \frac{(k-1)^2}{n^2} \right) = \pi \cdot \frac{2 \cdot k - 1}{n^2}$$

$$\text{ii. } P(R) = \frac{\text{area}(R)}{\pi} = \frac{\pi \cdot \frac{2 \cdot k - 1}{n^2}}{\pi} = \frac{2 \cdot k - 1}{n^2}$$

- Let's check if the sum fits the condition (sum = 1).

$$\sum_{k=1}^n \frac{2 \cdot k - 1}{n^2} = \frac{1}{n^2} \cdot \sum_{k=1}^n (2 \cdot k - 1) = \frac{1}{n^2} \cdot \frac{(2 \cdot n - 1 + 1) \cdot n}{2} = \frac{n^2}{n^2} = 1 - \text{Corresponds!}$$

- Probability of a random point:

$$P(R) = \frac{2 \cdot k - 1}{n^2}$$

- (d) Argue that the average case time complexity for your algorithm is $\Theta(n)$, relying on the result for standard BUCKET-SORT [CLRS, §8.4].

Answer:

- Since in (b) we calculated that it takes $\Theta(1)$ to write each element, then for all elements - $\Theta(n)$.
- Since, according to our algorithm, each coordinate is in one bucket, sorting each element takes $\Theta(1)$. Then for all coordinates it will be $\Theta(n)$.
- Writing our bucket into a new sorted array also takes $\Theta(n)$
- The result: $\Theta(n + n + n) = \Theta(n)$

3. Consider a modification of the MERGE-SORT algorithm [CLRS, §2.3] that stops recursion when the size of subarray becomes less than or equal to k ($k \leq n$). For arrays of size $\leq k$, the modified algorithm performs BUCKET-SORT [CLRS, §8.4] with a randomized QUICK-SORT [CLRS, §7.3] to sort each bucket. Answer the following questions about the modified algorithm. Provide justification (4–6 sentences) for each answer.

- (a) What is the worst case time complexity in terms of n and k ?

Answer: $\Theta(n \cdot \log \frac{n}{k} + n \cdot k)$

Justification:

The recursion of division into k parts takes $\Theta(\log \frac{n}{k})$ and for each division $\Theta(n)$ connections. In the worst case, for all $\frac{n}{k}$ subarrays, we have all the elements written to one bucket. Because of this, Quick-Sort spends k^2
 $\frac{n}{k} \cdot \Theta(k^2) = \Theta(n \cdot k)$ in the worst case.
 $\Theta(n \cdot \log \frac{n}{k}) + \Theta(n \cdot k) = \Theta(n \cdot \log \frac{n}{k} + n \cdot k)$

- (b) What is the best case time complexity in terms of n and k ?

Answer: $\Theta(n)$

Justification:

If our array is already sorted and we won't actually apply Merge-Sort when $n = k$:
 For Bucket-Sort, we will write, then if each bucket contains only one element, then Quick-Sort will be performed for all element in $\Theta(1) \cdot n = \Theta(n)$. Next, we assemble it into a new sorted array in $\Theta(n)$ time.

$$\Theta(n \cdot \log \frac{n}{n}) + \Theta(n) = \Theta(n)$$

- (c) What is the average³ case time complexity in terms of n and k ?

³assuming all elements in the input array are distinct and any initial order in the array is equally likely

Answer: $\Theta(n \cdot \log \frac{n}{k} + n \cdot \log k)$

Justification:

The recursion of division into k parts takes $\Theta(\log \frac{n}{k})$ and for each division $\Theta(n)$ connections.

In average, for all subarrays of size $\frac{n}{k}$, Quick-Sort spends an average time of $\Theta(k \cdot \log k)$

$\frac{n}{k} \cdot \Theta(k \cdot \log k) = \Theta(n \cdot \log k)$ in the average case.

$\Theta(n \cdot \log \frac{n}{k}) + \Theta(n \cdot \log k) = \Theta(n \cdot \log \frac{n}{k} + n \cdot \log k)$

The answer should be given using Θ -notation.

Provide a **brief** justification for each case (1–2 sentences).

References

- [CLRS] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms, Fourth Edition*. MIT press.